

第二次小作业

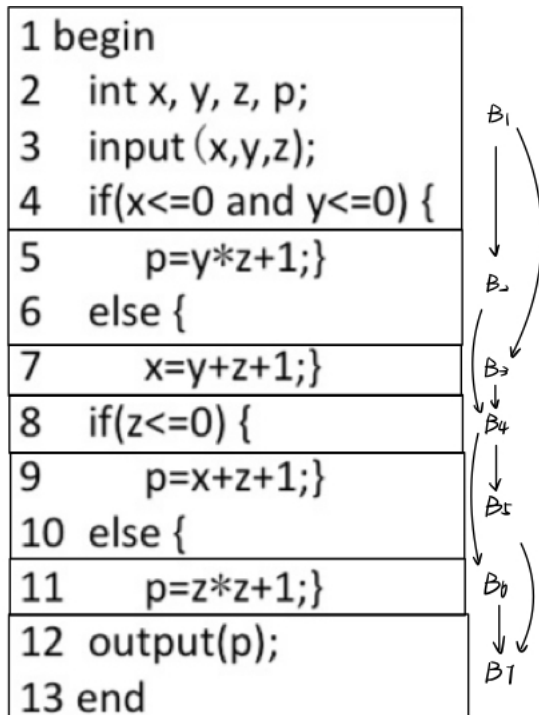
刘沛雨 2100012289

已知程序 P 如下图所示，其测试用例集 $T=\{t_1:\langle x=-1,y=-1,z=1\rangle,t_2:\langle x=2,y=12,z=0\rangle\}$ ，构造 P 的控制流图，

- (1) 计算 T 的语句覆盖、块覆盖、分支覆盖、判定覆盖、c-use 覆盖和 p-use 覆盖。
- (2) 使用大语言模型尝试解决该问题。

```
1 begin
2   int x, y, z, p;
3   input (x,y,z);
4   if(x<=0 and y<=0) {
5       p=y*z+1;}
6   else {
7       x=y+z+1;}
8   if(z<=0) {
9       p=x+z+1;}
10  else {
11      p=z*z+1;}
12  output(p);
13 end
```

- (1) 如下所示



语句覆盖: $t_1: \{1, 2, 3, 4, 5, 6, 8, 11, 12, 13\}$

$t_2: \{1, 2, 3, 4, 7, 8, 9, 10, 12, 13\}$

覆盖率: 1

块覆盖: $t_1: \{B_1, B_2, B_4, B_6, B_7\}$

$t_2: \{B_1, B_3, B_4, B_5, B_7\}$

覆盖率: 1

分支/判定覆盖

$t_1: \{B_1: true, B_4: false\}$

$t_2: \{B_1: false, B_4: true\}$

覆盖率: 1.

C-use: 未覆盖 $B_1 - B_2 - B_4 - B_5$, 覆盖率 90%.

P-use: 覆盖率: 1.

第一部分：构造程序 P 的控制流图 (CFG)

为了计算覆盖率，我们首先需要将代码划分为基本块（节点），并建立它们之间的执行顺序（边）。

节点定义：

- **N1** (第1-3行): `begin; int x,y,z,p; input(x,y,z);` (初始化与输入)
- **N2** (第4行): `if(x<=0 and y<=0)` (判定1)
- **N3** (第5行): `p=y*z+1;` (判定1为真的分支)
- **N4** (第7行): `x=y+z+1;` (判定1为假的分支)
- **N5** (第8行): `if(z<=0)` (判定2)
- **N6** (第9行): `p=x+z+1;` (判定2为真的分支)
- **N7** (第11行): `p=z*z+1;` (判定2为假的分支)
- **N8** (第12-13行): `output(p); end` (输出与结束)

控制流图的边：

- (N1, N2)
- (N2, N3) -> *True* 分支
- (N2, N4) -> *False* 分支
- (N3, N5)
- (N4, N5)
- (N5, N6) -> *True* 分支
- (N5, N7) -> *False* 分支
- (N6, N8)
- (N7, N8)

第二部分：测试用例集 T 的执行路径分析

我们有两个测试用例，先分析它们在控制流图上走过的路径：

1. 测试用例 $t_1 = \langle x = -1, y = -1, z = 1 \rangle$

- 到达 N2: $x \leq 0$ ($-1 \leq 0$, 真) 且 $y \leq 0$ ($-1 \leq 0$, 真) -> 判定为真
- 执行 N3: $p = (-1) \times 1 + 1 = 0$
- 到达 N5: $z \leq 0$ ($1 \leq 0$, 假) -> 判定为假
- 执行 N7: $p = 1 \times 1 + 1 = 2$
- t_1 执行路径: N1 -> N2 -> N3 -> N5 -> N7 -> N8

2. 测试用例 $t_2 = \langle x = 2, y = 12, z = 0 \rangle$

- 到达 N2: $x \leq 0$ ($2 \leq 0$, 假) -> 判定为假
- 执行 N4: $x = 12 + 0 + 1 = 13$
- 到达 N5: $z \leq 0$ ($0 \leq 0$, 真) -> 判定为真
- 执行 N6: $p = 13 + 0 + 1 = 14$
- t_2 执行路径: N1 -> N2 -> N4 -> N5 -> N6 -> N8

第三部分：各项覆盖率计算

根据上述路径，我们来计算各项覆盖率：

1. 语句覆盖 (Statement Coverage) 和 块覆盖 (Block Coverage)

- 定义：测试用例执行过的节点数 / 总节点数。
- t_1 覆盖了：N1, N2, N3, N5, N7, N8
- t_2 覆盖了：N1, N2, N4, N5, N6, N8
- 并集：{N1, N2, N3, N4, N5, N6, N7, N8}，所有节点均被覆盖。
- 结果：100% (8/8)

2. 分支覆盖 (Branch Coverage) / 判定覆盖 (Decision Coverage)

- 定义：测试用例执行过的判定分支数 / 总分支数。
- 判定点 N2: t_1 走了 True 分支， t_2 走了 False 分支。
- 判定点 N5: t_2 走了 True 分支， t_1 走了 False 分支。
- 所有 4 个分支均被覆盖。
- 结果：100% (4/4)

3. 数据流覆盖 (c-use 覆盖 和 p-use 覆盖)

数据流分析关注变量的定义 (def) 和 使用 (use)。

- c-use (计算使用): 变量在计算表达式或输出中被使用。
- p-use (谓词使用): 变量在条件判定中被使用。
- 覆盖率的分子是被测试覆盖的 `def -> clear path -> use` 对，分母是所有存在的有效对。

变量的定义 (Def):

- x : N1(输入), N4(赋值)
- y : N1(输入)
- z : N1(输入)
- p : N3(赋值), N6(赋值), N7(赋值)

(注: N3处定义的 p 是一个“死定义”, 因为无论后面走N6还是N7, p 都会被重新赋值, N3定义的 p 永远到不了N8的输出。因此不存在从N3出发的有效 c-use 对。)

计算 c-use 覆盖:

| 变量 | 对应定义 -> 计算使用对 | 经过路径 (Def-Clear) | 是否被 T 覆盖? |

| :--- | :--- | :--- | :--- |

| y | N1 -> N3 | N1 -> N2 -> N3 | 是 (t_1) |

| y | N1 -> N4 | N1 -> N2 -> N4 | 是 (t_2) |

| z | N1 -> N3 | N1 -> N2 -> N3 | 是 (t_1) |

| z | N1 -> N4 | N1 -> N2 -> N4 | 是 (t_2) |

| z | N1 -> N6 | N1 -> ... -> N6 | 是 (t_2) |

| z | N1 -> N7 | N1 -> ... -> N7 | 是 (t_1) |

| x | N1 -> N6 | N1 -> N2 -> N3 -> N5 -> N6 | 否 (T中没有这条路径) |

| x | N4 -> N6 | N4 -> N5 -> N6 | 是 (t_2) |

| p | N6 -> N8 | N6 -> N8 | 是 (t_2) |

| p | N7 -> N8 | N7 -> N8 | 是 (t_1) |

- 有效的 c-use 对共有 10 个。
- 测试集 T 覆盖了其中 9 个 (唯一未覆盖的是初始输入 x 的值直接流向 N6) 。
- **c-use 覆盖率: 90% (9/10)**

计算 p-use 覆盖:

| 变量 | 对应定义 -> 谓词使用对 | 目标分支 | 是否被 T 覆盖? |

| :--- | :--- | :--- | :--- |

| x | $N1 \rightarrow N2$ | True | 是 (t_1) |

| x | $N1 \rightarrow N2$ | False | 是 (t_2) |

| y | $N1 \rightarrow N2$ | True | 是 (t_1) |

| y | $N1 \rightarrow N2$ | False | 是 (t_2) |

| z | $N1 \rightarrow N5$ | True | 是 (t_2) |

| z | $N1 \rightarrow N5$ | False | 是 (t_1) |

- 有效的 p-use 路径共有 6 条。
- 测试集 T 覆盖了所有 6 条。
- **p-use 覆盖率: 100% (6/6)**

总结论:

该测试用例集的语句、块、判定（分支）和 p-use 覆盖率均达到了 **100%**。而 c-use 覆盖率由于缺失了一条无重新定义的数据流路径（N1 到 N6 的 x 流转），仅为 **90%**。